

# StayAnalytics — Database Design

---

# StayAnalytics — Database Design

---

**Group:** StayAnalytics **Course:** CSCI 4370 (Sami Menik, PhD)

Convert this Markdown file to **db\_design.pdf** before submission (e.g. `pandoc db_design.md -o db_design.pdf` ).

## 1. ER model

### Entity sets

| Entity    | Attributes (PK underlined in text)                                                                                                                                                                   |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| users     | <i>user_id</i> , username (UQ), email (UQ), password_hash, role, created_at                                                                                                                          |
| platform  | <i>platform_id</i> , name (UQ)                                                                                                                                                                       |
| genre     | <i>genre_id</i> , name (UQ)                                                                                                                                                                          |
| content   | <i>content_id</i> , title, type, country, language, release_year, duration_minutes, imdb_rating, votes, weighted_rating, engagement_score, popularity_score, trending_score, description, poster_url |
| review    | <i>review_id</i> , rating, comment, created_at                                                                                                                                                       |
| watchlist | (composite PK = user_id + content_id), status, added_at                                                                                                                                              |
| tag       | <i>tag_id</i> , name (UQ)                                                                                                                                                                            |

## Relationships

| Name                                              | Cardinality            | Participants                            |
|---------------------------------------------------|------------------------|-----------------------------------------|
| <code>belongs_to_platform</code>                  | N:1 (total on content) | content → platform                      |
| <code>belongs_to_genre</code>                     | N:1 (total on content) | content → genre                         |
| <code>writes_review</code>                        | 1:N                    | users → review                          |
| <code>is_about</code>                             | N:1                    | review → content                        |
| <code>watches</code> (M:N attribute relationship) | N:M                    | users ↔ content (with status, added_at) |
| <code>tagged_with</code> (M:N)                    | N:M                    | content ↔ tag                           |

## ER → table conversion

Each entity becomes a table with the listed PK. Each N:1 relationship becomes a foreign key on the “many” side ( `content.platform_id` , `content.genre_id` , `review.content_id` , `review.user_id` ). Each M:N relationship becomes a junction table:

- `watchlist` (`user_id`, `content_id`, `status`, `added_at`) — composite PK (`user_id`, `content_id`) . Carries the relationship attributes (status, added\_at).
- `content_tag` (`content_id`, `tag_id`) — composite PK.

We also added a UNIQUE constraint (`user_id`, `content_id`) on `review` so that “1 review per user per title” is enforced at the schema level. (Strictly speaking that turns review into a weak relationship- attribute, but the surrogate `review_id` simplifies INSERT/DELETE in JDBC.)

The resulting set of relations — eight in total — is exactly the schema implemented in `ddl.sql` .

## 2. Resulting relations

```
users      (user_id PK, username UQ, email UQ, password_hash, role, created_at)
platform   (platform_id PK, name UQ)
genre      (genre_id PK, name UQ)
content    (content_id PK,
            title, type, platform_id FK→platform, genre_id FK→genre,
            country, language, release_year, duration_minutes,
            imdb_rating, votes, weighted_rating,
            engagement_score, popularity_score, trending_score,
            description, poster_url)
review     (review_id PK,
            user_id FK→users, content_id FK→content,
            rating, comment, created_at,
            UQ(user_id, content_id))
watchlist  (user_id FK→users, content_id FK→content,
            status, added_at,
            PK(user_id, content_id))
tag         (tag_id PK, name UQ)
content_tag (content_id FK→content, tag_id FK→tag,
            PK(content_id, tag_id))
```

## 3. Functional dependencies

Starting from a hypothetical wide table

```
WIDE(content_id, title, type, platform_id, platform_name,
      genre_id, genre_name, country, language, release_year,
      duration_minutes, imdb_rating, votes, weighted_rating,
      engagement_score, popularity_score, trending_score,
      description, poster_url, tag_id, tag_name,
      user_id, username, email, password_hash, role, created_at,
      review_id, rating, comment, review_created_at,
      watchlist_status, watchlist_added_at)
```

we identified the following FDs:

| FD                                                                                                                                                                                                                                | Source                                    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| <code>content_id → title, type, platform_id, genre_id, country, language, release_year, duration_minutes, imdb_rating, votes, weighted_rating, engagement_score, popularity_score, trending_score, description, poster_url</code> | One canonical row per movie/series        |
| <code>platform_id → platform_name</code>                                                                                                                                                                                          | A platform name is determined by its id   |
| <code>genre_id → genre_name</code>                                                                                                                                                                                                | A genre name is determined by its id      |
| <code>tag_id → tag_name</code>                                                                                                                                                                                                    | A tag name is determined by its id        |
| <code>user_id → username, email, password_hash, role, created_at</code>                                                                                                                                                           | One row per user                          |
| <code>username → user_id</code>                                                                                                                                                                                                   | Username is unique → also a candidate key |
| <code>email → user_id</code>                                                                                                                                                                                                      | Email is unique → also a candidate key    |
| <code>review_id → user_id, content_id, rating, comment, review_created_at</code>                                                                                                                                                  | Surrogate review key                      |
| <code>(user_id, content_id) → review_id, rating, comment, review_created_at</code>                                                                                                                                                | “One review per (user, content)” rule     |
| <code>(user_id, content_id) → watchlist_status, watchlist_added_at</code>                                                                                                                                                         | Watchlist composite key                   |

## 4. Normalization

### 4.1 Why the wide table fails 3NF

In `WIDE`, `platform_name` depends on `platform_id`, which is a non-key attribute (the candidate key includes `content_id`). So `platform_id → platform_name` is a transitive dependency `content_id → platform_id → platform_name`, violating 3NF. The same applies to `genre_name`, `tag_name`, `username`, `email`, etc.

### 4.2 Decomposition steps

1. **Project users out of WIDE** using the FD `user_id → username, email, password_hash, role, created_at` → table `users`.

2. **Project platforms** using `platform_id → platform_name` → table `platform` .
3. **Project genres** using `genre_id → genre_name` → table `genre` .
4. **Project tags** using `tag_id → tag_name` → table `tag` .
5. **Project content** keyed on `content_id` → table `content` with FKs to platform/genre.
6. **Project reviews** keyed on `review_id` (with the UQ `(user_id, content_id)` ) → table `review` .
7. **Decompose the M:N watchlist** into table `watchlist` keyed on `(user_id, content_id)` .
8. **Decompose the M:N tagging** into `content_tag` keyed on `(content_id, tag_id)` .

### 4.3 Final relations are in BCNF

For every non-trivial FD listed above, the LHS is a superkey of the table it lives in:

- In `content` , the only non-trivial FDs come from `content_id` , which is the PK.
- In `platform` / `genre` / `tag` , the FD `*_id → name` (and `name → *_id` by uniqueness) — both sides are superkeys.
- In `users` , `user_id` is PK and `username` / `email` are also candidate keys (UNIQUE), so any FD whose LHS is one of those is superkey-driven.
- In `review` , both `review_id` (PK) and `(user_id, content_id)` (UNIQUE composite) are superkeys.
- `watchlist` and `content_tag` carry only their composite PKs, so any FD trivially has the PK as the LHS.

Therefore every relation is in **BCNF**, which implies **3NF** and removes the original transitive dependencies.

## 5. ER vs. normalization comparison

The ER-derived schema and the normalization-derived schema converge on the same eight tables. The only stylistic difference: the ER process leaves `review` with surrogate `review_id` , while pure decomposition would key it on `(user_id, content_id)` directly.

**We chose the ER schema** because

- the surrogate `review_id` simplifies INSERT/DELETE in JDBC (single- column AUTO\_INCREMENT key), and
- the additional `UNIQUE(user_id, content_id)` constraint preserves the same business invariant (“one review per user per title”) that the normalized form would have enforced through its key.

Both options are equivalent under BCNF.

## 6. Implementation references

- DDL: `ddl.sql`
- Sample data: `data.sql` (generated by `scripts/generate_movies_sql.py`)
- Indexes & timing: `perf.txt`
- Query catalog: `queries.sql`